Unified Detection of Tenant-Isolation Violations and QoS Fairness Breaches in Multi-Tenant Kubernetes

H S Jayanth
231IT024
IT418 Cloud Computing

Suvarna G
231IT077
IT418 Cloud Computing

***Abstract*—Multi-tenancy, in which many customers share one infrastructure pool, is central to cloud computing, yet Kubernetes runtime detection tools address only half the problem. A shared cluster can fail its tenants in two ways: an *isolation violation* (one tenant improperly reaching another's resources) and a *fairness/QoS violation* (one tenant's workload degrading a co-located tenant, the "noisy neighbor"). We build a unified runtime detection pipeline covering both: a control-plane monitor (Kubernetes audit log), a network-flow monitor (eBPF via Cilium/Hubble), and a resource monitor (kubelet/cAdvisor), fused by a deterministic correlation layer that declares a fairness violation only when resource pressure coincides with an independently measured victim-side latency breach. We evaluate on a three-tenant kind cluster with eight repeatable injection scenarios, five repetitions each, plus idle and benign-busy negative controls. Fusing control-plane and network-flow signals detects 25 of 25 isolation injections with no false positives, versus recall 0.60 and 0.40 for either alone. For fairness, the victim-side requirement removes the false alarms a pressure-only rule raises on legitimately busy tenants (precision 1.00 vs. 0.88 on a fresh validation run) and detects about six times sooner (2.5 s vs. 14.5 s), at a small recall cost (0.87 vs. 0.93). We also report where the result is fragile: it depends on one latency threshold that we selected post hoc and confirmed on independent runs.**

***Index Terms*—Kubernetes, multi-tenancy, tenant isolation, QoS fairness, noisy neighbor, eBPF, Cilium, runtime detection**

I. Introduction

Cloud computing is about sharing. A customer renting compute from a cloud provider does not get a dedicated machine, but a slice of a pool used by many others at the same time. Kubernetes makes this sharing work at scale by grouping workloads into namespaces and enforcing boundaries with RBAC, network policies and resource quotas.

Those boundaries are only as good as their configuration, and they fail in two very different ways. The first is a *security* failure: a misconfigured role binding gives a service account permissions it should not have, or a missing network policy lets traffic flow where it should not. The second is a *performance* failure: one tenant's workload consumes far more CPU or memory than intended, and the tenant running next to it sees slow responses. Nothing unauthorized happened and no security rule was broken, yet a tenant is harmed. This is the classic noisy-neighbor problem, and most security tooling ignores it.

This work builds and quantitatively evaluates a single detection pipeline for both failure modes on a multi-tenant Kubernetes cluster. Our contributions are:

- a three-source runtime monitoring pipeline (audit log, eBPF network flows, cgroup resource metrics) and a deterministic fusion layer that classifies each anomaly as isolation, fairness, or both;
- a fairness rule that requires an independent victim-side QoS breach, not just elevated usage, before declaring a violation;
- eight versioned, idempotent injection scenarios with ground-truth logging, and an evaluation harness with idle *and* benign-busy negative controls;
- an honest account of what the first evaluation got wrong (a fairness detector that scored zero, scoring artifacts, an over-sensitive threshold) and how each was fixed.

II. Literature Survey

A. Base paper: DAST for Kubernetes (FSE '25)

Shamim, Hu and Rahman evaluate three Dynamic Application Security Testing tools against a live Kubernetes deployment and find thousands of CIS-recommended configuration violations that static analysis alone missed [1]. Its central argument, that static checks cannot catch violations that only manifest at runtime, directly motivates this project. Its scope is limited, however: it documents findings but builds no detection tool; it considers only security misconfiguration, not resource fairness; it reports no precision, recall, false-positive rate or latency; and it relies on control-plane signals alone, missing violations visible only on the data or resource plane.

B. eBPF-based runtime security

Falco [5] and Cilium/Tetragon provide general-purpose eBPF runtime anomaly detection. Cilium, through Hubble [3], is the network-layer data source here. These tools treat runtime security generically: none frames detection around cross-tenant boundary violations, and none pairs security detection with a resource-fairness signal.

C. Resource fairness and noisy neighbors

Work on cloud resource fairness concentrates on scheduling and enforcement (cgroup throttling, Kubernetes QoS classes) rather than on correlating fairness breaches with an independent victim-side performance signal, and it does not connect to tenant-isolation detection.

D. The gap

We found no IEEE/ACM publication that unifies tenant-isolation security detection and tenant fairness/QoS detection under one fused pipeline evaluated quantitatively across both failure modes.

III. Motivation

Multi-tenancy is the concern, not a side issue. A provider's operational question is not "is this cluster misconfigured?" but "are my tenants safely and fairly sharing this pool?" Existing tooling answers only the security half.

The two violation types are invisible to each other's detectors. A control-plane monitor sees who asked to access what, but is blind to a CPU-hogging batch job that makes no unauthorized API call. An eBPF network monitor sees cross-namespace traffic, but not cgroup throttling, which lives in a different subsystem (kubelet/cAdvisor).

Claims need numbers. Without precision, recall, false-positive rate and detection latency measured against a structured scenario library and a static baseline, claims about detection effectiveness remain anecdotal.

IV. Objectives

The project has six objectives, built and validated in dependency order: (1) a multi-tenant kind testbed with enforced isolation and a static kube-bench baseline; (2) a control-plane monitor over the Kubernetes audit log; (3) data-plane (Hubble) and resource-plane (kubelet/cAdvisor) monitors; (4) a fusion and classification layer; (5) a CI/CD Trivy image-scan gate; (6) repeatable scenario libraries and a quantitative evaluation harness.

V. System Design

A. Architecture

Fig. 1 shows the pipeline. Three independent collectors feed one fusion layer. The data/resource plane is deliberately *two* collectors: Hubble's eBPF probes sit at the network/socket layer and cannot observe cgroup CPU throttling or memory pressure, which live in kubelet/cAdvisor. A k6 load generator supplies the independent victim-side signal.

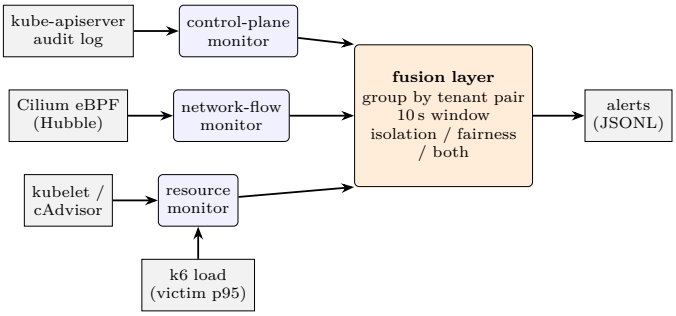


Fig. 1. System architecture: three monitors plus a victim-side QoS signal feed one fusion layer.

B. Monitors

The **control-plane monitor** tails the API-server audit log (Metadata level) and emits an event when a principal from one tenant's namespace acts on a resource scoped to another. The **network-flow monitor** wraps `hubble observe` and emits events for flows crossing tenant boundaries, including policy-denied egress whose peer identity Hubble does not resolve. The **resource monitor** polls kubelet/cAdvisor every 10 s and emits pressure events when a pod's CPU or memory usage reaches 85% of its limit or it is CFS-throttled. A **QoS monitor** tails k6 output, freezes a baseline p95 of `http_req_duration` over the first 10 s, and emits a `victim-qos-latency` breach event whenever the rolling 10 s p95 reaches a configurable multiple of that baseline (re-emitting every 5 s while the breach persists).

C. Fusion and classification

The fusion layer is rule-based and deterministic, which keeps results easy to validate end to end. Events are clustered per tenant pair using a 10 s sliding window.

1) **Isolation:** any cross-namespace control-plane or network-flow event is sufficient evidence on its own.
2) **Fairness:** resource pressure in one tenant's namespace *and* a victim-side QoS breach in a different tenant, overlapping in time. Elevated usage alone is never enough.
3) **Both:** an isolation and a fairness cluster for the same tenant pair that overlap in time are merged.

Each alert records the onset of its isolation evidence and of its fairness evidence separately, so that a merged alert can be scored per track (Section VIII-C).

D. CI/CD gate

A GitHub Actions workflow builds the demo image and fails the check on any HIGH or CRITICAL vulnerability with a known fix, using Trivy [6]. It is a static, pre-deploy complement to the runtime tracks.

VI. Testbed and Static Baseline

A. Testbed

A local kind cluster [7] (one control plane, two workers) runs Cilium [2] in place of the default CNI with

TABLE I
KUBE-BENCH CIS KUBERNETES 1.9 RESULTS

Category	PASS	FAIL	WARN	INFO
Master	40	7	12	0
etcd	7	0	0	0
Control Plane	1	0	4	0
Node	16	2	6	0
Policies	2	4	29	0
Total	66	13	51	0

Hubble enabled, and API-server audit logging wired to a host-readable file. A FastAPI demo application is deployed once per tenant namespace (`tenant-a`, `tenant-b`, `tenant-c`), each provisioned with a ServiceAccount and read-only Role/RoleBinding scoped to its own namespace; a default-deny Ingress/Egress NetworkPolicy with explicit same-namespace and DNS allows; and a ResourceQuota (`limits.cpu: 2`, `limits.memory: 1Gi`, `pods: 10`) with a LimitRange. These boundaries were verified before any monitor was built on them: a cross-namespace API call is denied, cross-namespace traffic is dropped, and an over-quota pod is rejected.

B. Static baseline: kube-bench

kube-bench [4] was run with the CIS 1.9 profile for Kubernetes 1.31 (Table I). Its weakest area for this project is Policies: 29 WARN, because NetworkPolicy coverage, RBAC scoping and namespace separation are marked Manual. kube-bench can note that a control should be checked but cannot confirm that the specific policies are correct and in force. Since it does not check ResourceQuota or LimitRange at all, a supplementary script confirms both exist for every tenant namespace: all three pass.

VII. Evaluation Methodology

A. Scenarios

Eight versioned, idempotent scenarios (each with inject and revert scripts and ground-truth logging) cover both tracks. *Isolation:* NetworkPolicy misconfiguration, RBAC over-permission, credential misuse, privilege escalation, and a data-plane-only crossing (traffic with no API-server footprint). *Fairness:* a CPU-hog batch job, memory pressure, and a ResourceQuota bypass. Each runs five times (25 isolation and 15 fairness runs). All fairness hogs run in `tenant-a`, on the node hosting the victim `tenant-b`.

B. Controls

False-positive rate is ill-posed without explicit no-injection windows. We use ten: five *idle* windows, and five *benign-busy* windows in which a well-behaved tenant saturates its own 200m CPU limit with a single small job. Idle windows cannot separate a naive pressure-only rule from the fused rule (neither fires on silence); benign-busy windows can.

TABLE II
DETECTOR CONFIGURATIONS OVER THE FULL SWEEP (FP SHOWN AS IDLE/BUSY WINDOWS). THE 1.5× FUSED-FAIRNESS ROW IS THE INITIAL THRESHOLD; SEE TABLE III.

Config	Track	TP	FN	FP	Prec.	Recall	Lat. (s)
control-plane-only	iso.	15	10	0/0	1.00	0.60	0.22
network-flow-only	iso.	10	15	0/0	1.00	0.40	0.70
fused-isolation	iso.	25	0	0/0	1.00	**1.00**	0.41
pressure-only	fair.	15	0	0/3	0.83	1.00	13.45
fused-fairness (1.5×)	fair.	13	2	0/5	0.72	0.87	2.48

C. Detector configurations

Because configurations differ only in how the same logs are classified, each scenario runs once and is re-scored under every configuration. Isolation: *control-plane-only*, *network-flow-only*, *fused-isolation*. Fairness: *pressure-only* (elevated usage alone is sufficient: the naive baseline) and *fused-fairness* (requires the victim-side breach).

D. Metrics

An alert matches a run if it is of the right track, involves at least two expected tenants (one for fairness), and its evidence for that track begins within the injection window plus the 10 s correlation slack. Precision and false-positive rate are computed against the negative-control windows; recall against the injections; detection latency is injection start to the first matching evidence.

E. Creating real contention

On our 16-core host, hogs limited to 200m–1.5 cores did not measurably affect a co-located `/healthz` handler: victim p95 stayed near 3.6 ms before and during every injection, so there was no noisy-neighbor effect to detect. We therefore pin the worker node to a 2-CPU cpuset, schedule hogs on the victim's node, and scale them (six 200m CPU hogs; six memory-ramping pods that keep rewriting their working set; a multi-process hog under a 1200m limit for quota bypass). This is engineered contention: per-container CPU limits protected co-located tenants well on a large host.

VIII. Results

A. Main sweep

Table II gives the sweep: 8 scenarios × 5 repetitions plus ten controls, with the victim breach ratio at 1.5×.

Isolation: the two signals are complementary. Control-plane-only misses every run of NetworkPolicy misconfiguration and data-plane-only crossing (there is no API call to observe). Network-flow-only misses every run of RBAC over-permission, credential misuse and privilege escalation (no cross-tenant data-plane flow). Fusing them reaches 25/25 with no false positives. A static baseline sees none of these runtime events.

Fairness at 1.5× is worse than the naive rule on precision. Benign-busy windows reveal that a single 200m

TABLE III
POST-HOC REPLAY OF FUSED-FAIRNESS AT SEVERAL BREACH RATIOS (15 INJECTIONS; 5 IDLE, 5 BUSY CONTROLS).

Ratio	TP	FN	FP idle/busy	Prec.	Lat. (s)
1.5	12	3	0/5	0.71	2.49
1.75	15	0	0/5	0.75	2.64
2.0	15	0	0/4	0.79	2.77
2.5	15	0	0/0	1.00	4.19
3.0	15	0	0/0	1.00	4.73

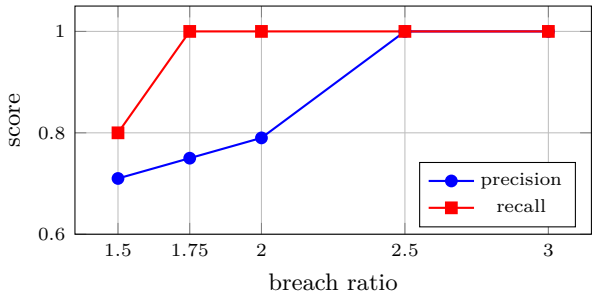


Fig. 2. Fused-fairness precision and recall vs. victim breach ratio (post-hoc replay).

job raises victim p95 to 7–9 ms against 5 ms idle (1.5–1.8×), so the fused rule fires on all five. Idle windows alone hid this.

B. Threshold sensitivity and validation

We replayed the sweep's k6 output through the live detector logic at several breach ratios (Table III, Fig. 2); the replay reproduces the live 1.5× outcome. At 2.5× every false alarm disappears while recall is retained. Because this selects the operating point on the data it is scored on, we validated it on a *fresh live run* (new injections and controls) with 2.5× (Table IV). Fused fairness then has no false positives where the naive rule has two, and detects about six times sooner, because kubelet/cAdvisor-derived pressure events arrive tens of seconds after a hog starts whereas the k6 signal is near real time. The price is recall: 0.87 against 0.93, since the fused rule fires only if the victim is actually hurt.

C. Evaluation bugs found and fixed

Our first sweep scored fused fairness at 0 of 15, and later analysis uncovered several problems:

- **No victim effect.** The fusion rule was correct, but its required evidence never existed (Section VII-E). Fixed by the node CPU pin and scaled hogs.
- **Edge-triggered QoS events.** One event per breach onset, plus lingering contention between back-to-back runs, produced a single event for a whole sweep. The monitor now re-emits while a breach persists, and the harness waits 45 s after fairness runs (20 s otherwise).
- **Merged alerts hid detections.** An alert's first-observed time was the earliest evidence of either kind, so lingering fairness evidence merging into an isolation alert pushed its start before the injection window, scoring it a miss (three isolation misses in the second sweep). Alerts now carry per-track onsets; this took fused-isolation from 22/25 to 25/25.
- **A meaningless metric.** "Correlation overhead" measured the time between an event and scoring, about 330 s, not fusion cost. It is replaced by measured classifier runtime (about 0.02 s for 1,300 events).
- **Flaky tooling.** Version checks using `grep -q` under `pipefail` intermittently reported tools as missing; fixed.

TABLE IV
FRESH VALIDATION RUN AT RATIO 2.5× (15 INJECTIONS; 5 IDLE, 5 BUSY CONTROLS).

Config	TP	FN	FP idle/busy	Prec.	Recall	Lat. (s)
pressure-only	14	1	0/2	0.88	0.93	14.54
fused-fairness	13	2	0/0	**1.00**	0.87	**2.54**

IX. DISCUSSION AND LIMITATIONS

What the evidence supports. Signals from different planes catch different violations, so fusion materially improves isolation recall. The fairness rule's value is precision and latency, and it depends on one tunable.

Limitations.

- *Remaining misses.* In validation, fused fairness missed two of 15 CPU-hog runs: in one the resource monitor never reached its 85% threshold; in the other an earlier breach merged into the alert so it began before the injection window.
- *Post-hoc threshold.* The 2.5× ratio was chosen after seeing data and confirmed on one fresh run, not cross-validated. With 15 runs per condition, intervals are wide.
- *Engineered contention, single host, one trivial victim workload* (`/healthz` through a port-forward). Results may not transfer to production multi-node clusters.
- *Unaccounted alerts.* Alerts matching no run are reported in the output but not counted as false positives; many are repeat alerts for the same cluster, but we have not audited them, so precision is measured against the control windows only.
- *Operational dependencies.* The audit log is root-owned on the host, requiring a workaround for the monitor to read it.
- *Scope.* Automatic remediation was an accepted stretch goal and is not implemented.

X. CONCLUSION

We built a unified runtime detection pipeline for tenant isolation and QoS fairness in Kubernetes and evaluated it on eight scenarios with idle and benign-busy controls. Fusing control-plane and network-flow evidence detects all 25 isolation injections with no false positives, where each alone detects 60% or 40%. Requiring a victim-side QoS

breach for fairness eliminates false alarms on legitimately busy tenants and cuts detection latency about six times, at a small recall cost, provided the breach threshold is set above the noise of a lightly loaded victim. The negative result on our first run, and the analysis that explained it, are as informative as the headline numbers. Future work includes cross-validated thresholds, multi-node and more realistic victim workloads, auditing unaccounted alerts, and the remediation stretch goal.

References

[1] S. I. Shamim, H. Hu, and A. Rahman, "Dynamic Application Security Testing for Kubernetes Deployment: An Experience Report from Industry," in *Proc. ACM Int. Conf. Foundations of Software Engineering (FSE)*, 2025.

[2] Cilium Authors, *Cilium: eBPF-based Networking, Observability, and Security*, 2024. [Online]. Available: https://cilium.io

[3] Cilium Authors, *Hubble: Network, Service & Security Observability for Kubernetes*, 2024. [Online]. Available: https://github.com/cilium/hubble

[4] Aqua Security, *kube-bench*, 2024. [Online]. Available: https://github.com/aquasecurity/kube-bench

[5] Falco Authors, *Falco: Cloud Native Runtime Security*, 2024. [Online]. Available: https://falco.org

[6] Aqua Security, *Trivy: Vulnerability Scanner for Containers*, 2024. [Online]. Available: https://github.com/aquasecurity/trivy

[7] The Kubernetes Authors, *Kubernetes Documentation*, 2024. [Online]. Available: https://kubernetes.io/docs